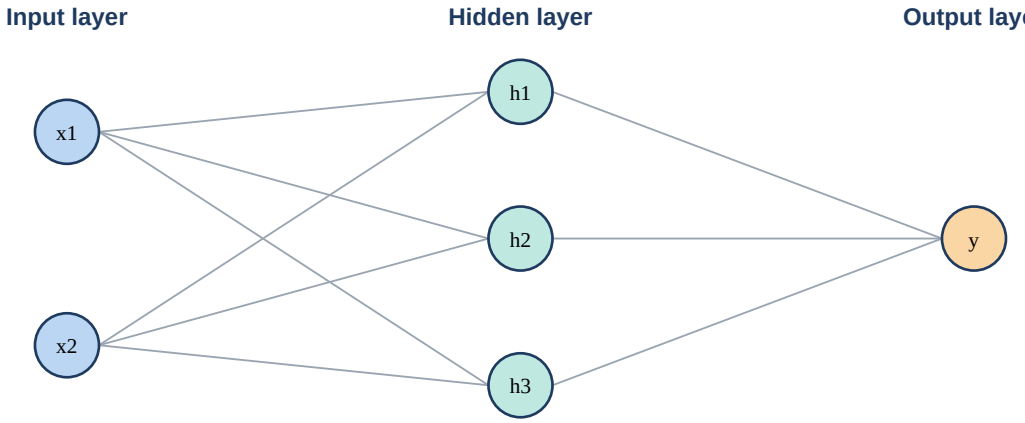
Artificial Neural Networks

Complete Interview Notes with Worked Examples



Every neuron is connected to every neuron in the next layer (fully connected / dense)

Inside
1. What is an ANN? 2. The neuron 3. Architecture 4. Activation functions
5. Forward propagation (worked example) 6. Loss functions 7. Backpropagation (worked example)
8. Gradient descent & optimizers 9. Overfitting & regularization 10. Vanishing gradients & initialization
11. Hyperparameters 12. Code (NumPy + Keras) 13. ANN vs CNN vs RNN 14. 30 interview Q&As 15. Cheat sheet

1. What is an Artificial Neural Network?

An **Artificial Neural Network (ANN)** is a machine-learning model made of layers of simple computing units (*neurons*) that learn a mapping from inputs to outputs by adjusting numeric **weights** from data. It is loosely inspired by the brain, but in practice it is just a big, differentiable function **y = f(x; W)** whose parameters W are tuned to minimize an error.

One-line interview answer: *An ANN is a stack of weighted sums followed by non-linear activations, trained with backpropagation and gradient descent to minimize a loss function.*

Why do we need them?

- Linear models cannot learn non-linear patterns (e.g. XOR). Hidden layers with non-linear activations can.
- **Universal Approximation Theorem:** a network with one hidden layer and enough neurons can approximate any continuous function to arbitrary accuracy.
- They learn features automatically from raw data instead of needing hand-crafted features.

Biological vs artificial neuron

Biological	Artificial
Dendrites receive signals	Inputs x1..xn
Synapse strength	Weights w1..wn
Cell body sums signals	Weighted sum z = w.x + b
Fires if threshold crossed	Activation function a = g(z)
Axon sends output	Output passed to next layer

2. The Artificial Neuron (Perceptron)

A single neuron does two things:

- **Linear step:** z = w1*x1 + w2*x2 + ... + wn*xn + b (weighted sum plus **bias**)
- **Non-linear step:** a = g(z), where g is the activation function

The **bias** shifts the activation curve left/right, so a neuron can fire even when all inputs are zero (like the intercept in y = mx + c).

Tiny example: will I go to the party?

Inputs: x1 = weather is good (1/0), x2 = friends going (1/0). Weights: w1 = 2, w2 = 3, bias b = -4. Using a step activation (output 1 if z >= 0):

x1	x2	z = 2*x1 + 3*x2 - 4	Output
0	0	-4	0 (stay home)
1	0	-2	0
0	1	-1	0
1	1	+1	1 (go!)

This neuron has learned an **AND** gate. A single perceptron can only learn **linearly separable** problems; it cannot learn XOR. That limitation is why we stack neurons into multiple layers (MLP).

3. Architecture of an ANN

Layer	Role	Notes
Input layer	Receives raw features	One neuron per feature. No computation or activation.
Hidden layer(s)	Learn intermediate representations	More layers = deeper network (deep learning). Usually ReLU.
Output layer	Produces the prediction	1 neuron + sigmoid (binary), N neurons + softmax (multi-class), 1 neuron linear (regression).

Parameters count example: network 4 -> 5 -> 3. Layer 1: 4*5 weights + 5 biases = 25. Layer 2: 5*3 + 3 = 18. Total = **43 trainable parameters**. Formula per dense layer: (inputs x outputs) + outputs.

Types of ANN: Feed-forward / MLP (information flows one way), CNN (images), RNN/LSTM/GRU (sequences), Autoencoders, GANs, Transformers.

4. Activation Functions

Without a non-linear activation, any number of stacked layers collapses into one linear transformation. Activations give the network its power to model curves.

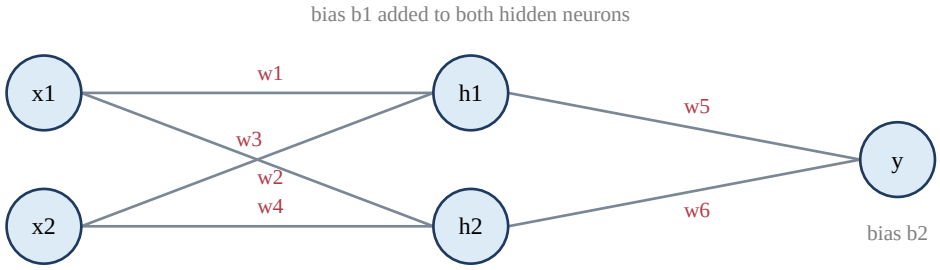
Function	Formula	Range	Use / Pros / Cons
Sigmoid	1 / (1 + e^-z)	(0, 1)	Binary output layer. Smooth, but saturates (vanishing gradient), not zero-centered.
Tanh	(e^z - e^-z)/(e^z + e^-z)	(-1, 1)	Zero-centered, better than sigmoid in hidden layers, still saturates.
ReLU	max(0, z)	[0, inf)	Default for hidden layers. Fast, no saturation for z>0. Risk: *dying ReLU* (neuron stuck at 0).
Leaky ReLU	max(0.01z, z)	(-inf, inf)	Fixes dying ReLU by allowing a small negative slope.
Softmax	e^zi / sum(e^zj)	(0, 1), sums to 1	Multi-class output layer: gives class probabilities.
Linear	z	(-inf, inf)	Regression output layer.

Derivatives you must remember

- Sigmoid: **g'(z) = g(z)(1 - g(z))**, maximum value 0.25 (the root of vanishing gradients).
- Tanh: g'(z) = 1 - g(z)^2, maximum value 1.
- ReLU: g'(z) = 1 if z > 0 else 0.

5. Forward Propagation (Worked Example)

Forward propagation pushes inputs through the network layer by layer to compute a prediction: for each layer, **z = W.a_prev + b**, then **a = g(z)**.



Setup (2-2-1 network, sigmoid everywhere, target t = 1.0): inputs x1 = 0.05, x2 = 0.10. Weights w1 = 0.15, w2 = 0.20, w3 = 0.25, w4 = 0.30, b1 = 0.35; w5 = 0.40, w6 = 0.45, b2 = 0.60.

Step 1 - hidden layer

```
z_h1 = w1*x1 + w2*x2 + b1 = 0.15*0.05 + 0.20*0.10 + 0.35 = 0.3775
h1   = sigmoid(0.3775) = 0.5933

z_h2 = w3*x1 + w4*x2 + b1 = 0.25*0.05 + 0.30*0.10 + 0.35 = 0.3925
h2   = sigmoid(0.3925) = 0.5969
```

Step 2 - output layer

```
z_o = w5*h1 + w6*h2 + b2 = 0.40*0.5933 + 0.45*0.5969 + 0.60 = 1.1059
y   = sigmoid(1.1059) = 0.7514
```

Step 3 - compute the loss (squared error, with the 1/2 for a clean derivative)

```
L = 1/2 * (t - y)^2 = 0.5 * (1.0 - 0.7514)^2 = 0.030910
```

The network predicts **0.751** but we wanted **1.0**, so the error is 0.0309. Backpropagation will now tell each weight how to change to shrink it.

6. Loss Functions

Task	Loss	Formula	Output activation
Regression	MSE	(1/n) sum (y - yhat)^2	Linear
Regression (outliers)	MAE / Huber	(1/n) sum |y - yhat|	Linear
Binary classification	Binary cross-entropy	-[y log(p) + (1-y) log(1-p)]	Sigmoid
Multi-class	Categorical cross-entropy	- sum y_k log(p_k)	Softmax

Why cross-entropy for classification? It penalizes confident wrong answers very heavily (log of a tiny probability is a large negative number) and, combined with sigmoid/softmax, gives a clean gradient (p - y) that does not vanish when the neuron saturates, unlike MSE.

> **Interview tip:** Loss = error on *one* sample. Cost function = average loss over the dataset/batch. Many people use the words interchangeably, but mention the distinction to sound precise.

7. Backpropagation (Worked Example)

Backpropagation applies the **chain rule** of calculus to compute the gradient of the loss with respect to every weight, moving from the output layer backwards. Then gradient descent updates the weights: **w_new = w_old - learning_rate * dL/dw**.

Chain rule for output weight w5:

```
dL/dw5 = dL/dy * dy/dz_o * dz_o/dw5
       = (y - t) * y(1 - y) * h1
```

Using our numbers

```
dL/dy      = y - t            = 0.7514 - 1.0          = -0.2486
dy/dz_o    = y(1 - y)         = 0.7514 * 0.2486     = 0.1868
delta_o    = (y-t) * y(1-y)   = -0.046449

dL/dw5 = delta_o * h1 = -0.046449 * 0.5933 = -0.027557
dL/dw6 = delta_o * h2 = -0.046449 * 0.5969 = -0.027725
dL/db2 = delta_o                          = -0.046449
```

Pushing the error back to the hidden layer (for w1):

```
dL/dw1 = delta_o * w5 * h1(1 - h1) * x1
delta_h1 = -0.046449 * 0.40 * 0.2413 = -0.004483
dL/dw1 = delta_h1 * x1 = -0.004483 * 0.05 = -0.0002242
(w2, w3, w4 follow the same pattern: delta_hidden * its input)
```

Weight update with learning rate = 0.5

Weight	Old	Gradient	New = old - 0.5*grad
w5	0.4000	-0.027557	0.41378
w6	0.4500	-0.027725	0.46386
b2	0.6000	-0.046449	0.62322
w1	0.1500	-0.0002242	0.150112
w2	0.2000	-0.0004483	0.200224
w3	0.2500	-0.0002515	0.250126
w4	0.3000	-0.0005029	0.300251

Result of one training step: repeating the forward pass with the new weights gives y = **0.7589** (was 0.7514) and loss = **0.029068** (was 0.030910). The loss dropped, so the network moved in the right direction. Training = repeating this thousands of times.

The 4 steps of every training iteration

- **1. Forward pass** - compute predictions.
- **2. Compute loss** - compare with true labels.
- **3. Backward pass** - compute gradients via chain rule.
- **4. Update** - adjust weights with an optimizer.

Note: for b1 the gradient is the sum of delta_h1 and delta_h2 because the same bias feeds both hidden neurons. Shared parameters accumulate gradients from every path they influence.

8. Gradient Descent and Optimizers

Imagine standing on a foggy hillside and wanting to reach the valley (minimum loss). You feel the slope under your feet (the gradient) and step downhill. The **learning rate** is your step size.

Variant	Data per update	Pros	Cons
Batch GD	Whole dataset	Stable, smooth	Slow, memory heavy
Stochastic GD (SGD)	1 sample	Fast updates, can escape local minima	Very noisy
Mini-batch GD	32-256 samples	Best balance; GPU friendly (the standard)	Needs batch-size tuning

Learning rate effects

- **Too high:** loss oscillates or diverges (overshoots the valley).
- **Too low:** painfully slow training, may get stuck.
- Fix: learning-rate schedules / decay, warm-up, or adaptive optimizers.

Popular optimizers

Optimizer	Idea
SGD + Momentum	Keeps a running velocity of past gradients so it speeds along consistent directions and dampens zig-zags.
RMSprop	Divides the step by a running average of squared gradients: per-parameter adaptive learning rate.

Optimizer	Idea
Adam	Momentum + RMSprop with bias correction. Default choice (lr = 0.001, beta1 = 0.9, beta2 = 0.999).
AdamW	Adam with decoupled weight decay; common for transformers.

9. Overfitting and Regularization

Overfitting: the model memorizes the training data (very low train error) but fails on unseen data (high validation error). **Underfitting:** the model is too simple and performs badly on both.

Symptom	Train error	Validation error	Fix
Underfit (high bias)	High	High	Bigger model, more features, train longer, less regularization
Good fit	Low	Low (close to train)	-
Overfit (high variance)	Very low	High	More data, regularization, dropout, early stopping, simpler model

Regularization techniques

Technique	How it works
L2 (weight decay)	Adds lambda*sum(w^2) to the loss; keeps weights small and smooth.
L1	Adds lambda*sum|w|; pushes some weights to exactly 0 (sparsity).
Dropout	Randomly turns off a fraction (e.g. 20-50%) of neurons each training step so neurons cannot co-adapt. Disabled at inference.
Early stopping	Stop training when validation loss stops improving.
Data augmentation	Create modified copies of data (flip, crop, noise).
Batch Normalization	Normalizes layer inputs per mini-batch; stabilizes and speeds training, mild regularizing effect.

10. Vanishing / Exploding Gradients and Weight Initialization

Vanishing gradient: during backprop, gradients are multiplied layer after layer. With sigmoid (max derivative 0.25), ten layers shrink the gradient to about 0.25^10 = 0.000001, so early layers barely learn. **Exploding gradient:** if the factors are greater than 1, gradients blow up to huge values (NaN loss).

Problem	Solutions
Vanishing	ReLU / Leaky ReLU, proper initialization, batch norm, residual (skip) connections, LSTM/GRU for sequences
Exploding	Gradient clipping, smaller learning rate, proper initialization, batch norm

Weight initialization: never initialize all weights to zero - every neuron in a layer would compute the same thing and receive the same gradient (the *symmetry problem*), so they never differentiate. Use **Xavier/Glorot** init for sigmoid/tanh (variance = 2/(n_in + n_out)) and **He init** for ReLU (variance = 2/n_in).

11. Hyperparameters

Parameters are *learned* (weights, biases). Hyperparameters are *set by you* before training:

Hyperparameter	Typical choices / advice
Learning rate	Most important. Try 1e-1 to 1e-5 on a log scale.
Number of layers / neurons	Start small, grow until it can overfit, then regularize.
Batch size	32, 64, 128, 256. Larger = stable but may generalize worse.
Epochs	Use early stopping instead of guessing.
Activation functions	ReLU in hidden layers by default.
Optimizer	Adam as the default starting point.
Dropout rate / L2 lambda	0.2-0.5 / 1e-4 to 1e-2

Tuning methods: grid search, random search (often better), Bayesian optimization (Optuna, Keras Tuner).

Epoch vs batch vs iteration: 1 epoch = one full pass over the data. If you have 1000 samples and batch size 100, one epoch = 10 iterations (10 weight updates).

12. Code Examples

A. Neural network from scratch in NumPy (learns XOR)

```
import numpy as np
np.random.seed(1)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])                 # XOR: not linearly separable

sigmoid  = lambda z: 1 / (1 + np.exp(-z))
W1 = np.random.randn(2, 4); b1 = np.zeros((1, 4))   # 2 inputs -> 4 hidden
W2 = np.random.randn(4, 1); b2 = np.zeros((1, 1))   # 4 hidden -> 1 output
lr = 0.5

for epoch in range(10000):
    # forward
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    # backward (MSE loss)
    d_out = (out - y) * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    # update
    W2 -= lr * h.T @ d_out;  b2 -= lr * d_out.sum(0, keepdims=True)
    W1 -= lr * X.T @ d_h;    b1 -= lr * d_h.sum(0, keepdims=True)

print(out.round(3))     # approx [[0],[1],[1],[0]]
```

B. Same idea in Keras (production style)

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(20,)),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid"),       # binary classification
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

early = keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)
model.fit(X_train, y_train, validation_split=0.2,
          epochs=100, batch_size=32, callbacks=[early])
model.evaluate(X_test, y_test)
```

Practical checklist: scale/standardize inputs (neural nets are sensitive to feature scale), split into train/validation/test, monitor validation loss, and try to overfit a tiny subset first to confirm the code works.

13. ANN vs CNN vs RNN

	ANN / MLP	CNN	RNN / LSTM
Best for	Tabular data	Images, video, spatial data	Sequences: text, time series
Key idea	Fully connected layers	Convolution filters share weights, detect local patterns	Hidden state carries memory across time steps
Parameters	Many (every input connects to every neuron)	Fewer (weight sharing)	Shared across time steps
Order awareness	None	Spatial locality	Temporal order
Weakness	Ignores structure; poor on raw images	Not natural for sequences	Slow, long-range memory issues (Transformers now preferred)

ANN vs traditional ML: ANNs shine with large data and unstructured inputs (images, audio, text) and learn features automatically, but need more data and compute and are less interpretable. For small tabular data, gradient boosting (XGBoost) often wins.

14. Interview Questions and Answers

Q1. What is an ANN?

A layered network of neurons that computes weighted sums plus non-linear activations and learns its weights from data using backpropagation and gradient descent.

Q2. Why do we need activation functions?

Without non-linearity, stacked layers reduce to a single linear model. Activations let the network learn complex, non-linear relationships.

Q3. What is the role of bias?

It shifts the activation function so the neuron can output non-zero values when inputs are zero; it adds flexibility, like an intercept.

Q4. What is backpropagation?

An efficient algorithm that uses the chain rule to compute the gradient of the loss with respect to every weight, from the output layer back to the input layer; an optimizer then uses these gradients to update the weights.

Q5. Difference between a perceptron and an MLP?

A perceptron is a single neuron (linear classifier only). An MLP has one or more hidden layers with non-linear activations and can learn non-linear boundaries like XOR.

Q6. Why is ReLU preferred over sigmoid in hidden layers?

It is cheap to compute, does not saturate for positive inputs (so gradients do not vanish), and converges faster. Downside: dying ReLU, fixed by Leaky ReLU.

Q7. What is the vanishing gradient problem?

Gradients shrink exponentially as they propagate back through many layers (esp. with sigmoid/tanh), so early layers learn very slowly. Fix: ReLU, good init, batch norm, residual connections.

Q8. What is the exploding gradient problem?

Gradients grow exponentially and cause unstable updates or NaN. Fix: gradient clipping, lower learning rate, better initialization.

Q9. Why not initialize all weights to zero?

All neurons in a layer would get identical gradients and stay identical (symmetry), so the layer behaves as one neuron. Use random init (Xavier/He).

Q10. Explain overfitting and how to prevent it.

The model memorizes training data and fails to generalize. Prevent with more data, augmentation, L1/L2, dropout, early stopping, simpler architecture.

Q11. How does dropout work?

During training each neuron is randomly dropped with probability p, forcing redundancy and reducing co-adaptation. At inference all neurons are used (weights scaled appropriately).

Q12. What is batch normalization?

It normalizes activations of a layer using batch mean and variance, then rescales with learned parameters. It stabilizes training, allows higher learning rates and slightly regularizes.

Q13. Epoch, batch, iteration?

Epoch = one pass over all data; batch = subset of samples used for one update; iteration = one update step. Iterations per epoch = N / batch size.

Q14. Batch GD vs SGD vs mini-batch?

Batch uses all data per update (stable, slow); SGD uses one sample (noisy, fast); mini-batch uses small groups and is the standard compromise.

Q15. What does the learning rate do?

It scales the gradient step. Too high causes divergence, too low causes slow convergence.

Q16. Why is Adam popular?

It combines momentum and per-parameter adaptive learning rates, works well with little tuning.

Q17. MSE vs cross-entropy?

MSE for regression. Cross-entropy for classification because it penalizes confident mistakes and gives non-vanishing gradients with sigmoid/softmax.

Q18. Sigmoid vs softmax?

Sigmoid gives an independent probability for one class (binary or multi-label). Softmax gives a probability distribution over mutually exclusive classes that sums to 1.

Q19. What is the Universal Approximation Theorem?

A feed-forward network with a single hidden layer and enough neurons can approximate any continuous function. It says nothing about how easily it can be learned, which is why depth helps in practice.

Q20. Why is feature scaling important?

Features on different scales distort gradients and slow convergence. Standardize or normalize inputs.

Q21. How do you decide the number of layers and neurons?

Start simple, increase capacity until the model can overfit the training set, then add regularization. Use validation performance and hyperparameter search.

Q22. Parameters vs hyperparameters?

Parameters (weights, biases) are learned from data. Hyperparameters (learning rate, layers, batch size) are set before training.

Q23. What is a deep neural network?

A network with multiple hidden layers (commonly more than 2-3), enabling hierarchical feature learning.

Q24. How do you handle class imbalance in an ANN?

Class weights in the loss, oversampling/undersampling, focal loss, and evaluating with precision/recall/F1/AUC rather than accuracy.

Q25. What is early stopping?

Monitor validation loss and stop (restoring the best weights) once it stops improving for a set number of epochs (patience).

Q26. What is a dead ReLU?

A neuron whose input is always negative outputs 0 with zero gradient, so it never recovers. Caused by high learning rates or bad init; use Leaky ReLU/He init.

Q27. How does a model with a loss that does not decrease get debugged?

Check data/labels and scaling, verify the loss on a tiny subset can reach ~0, lower the learning rate, check gradient flow and activation choice, confirm the output activation matches the loss.

Q28. Can an ANN work with a very small dataset?

It is risky: it overfits easily. Use a smaller network, strong regularization, augmentation, transfer learning, or prefer simpler models.

Q29. Why are neural networks called black boxes?

Thousands to millions of weights make individual decisions hard to interpret. Tools like SHAP, LIME, saliency maps help explain them.

Q30. What is transfer learning?

Reuse a model pre-trained on a large dataset and fine-tune it on your smaller task, saving data and compute.

Q31. What is the difference between a loss function and an activation function?

Activation is applied inside the network to add non-linearity to each neuron. Loss is applied at the end to measure prediction error and drive learning.

15. One-Page Cheat Sheet

Concept	Remember
Neuron	a = g(w.x + b)
Forward pass	z = W.a_prev + b, then a = g(z), layer by layer
Hidden activation	ReLU (default); tanh/sigmoid are older choices
Output + loss	Regression: linear + MSE. Binary: sigmoid + BCE. Multi-class: softmax + categorical CE
Backprop	Chain rule, output to input. delta_out = dL/dz; dL/dW = delta * a_prev
Update rule	w = w - lr * dL/dw
Sigmoid derivative	g(1 - g), max 0.25
Parameter count (dense)	(n_in * n_out) + n_out
Optimizer default	Adam, lr = 0.001
Init	He for ReLU, Xavier for tanh/sigmoid, never zeros
Overfitting fixes	More data, dropout, L2, early stopping, augmentation, batch norm
Vanishing gradient fixes	ReLU, init, batch norm, residual connections, LSTM
Always	Scale inputs, split data, watch validation loss

How to answer ANN questions in an interview: (1) give a one-line definition, (2) explain the intuition with a tiny example, (3) mention the formula, (4) state the trade-off or limitation, (5) connect to a real project. Interviewers reward clarity and trade-off awareness more than memorized jargon.